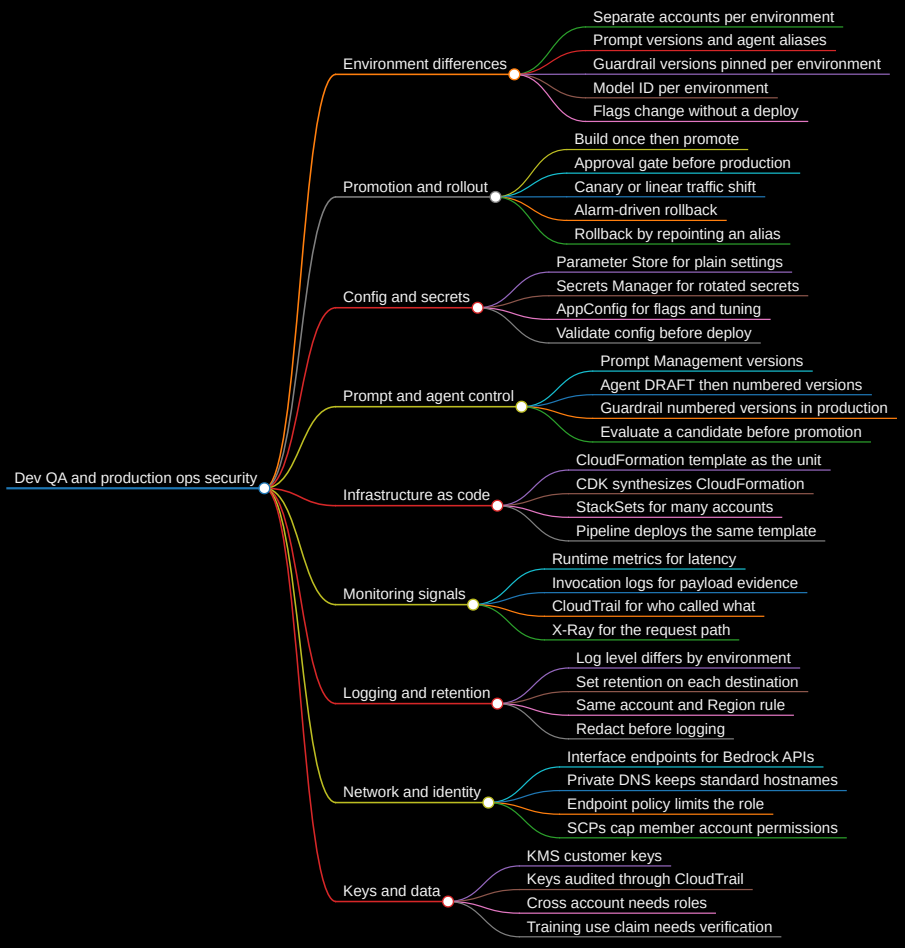
Module m7: Dev, QA, and production, and how to run and secure a GenAI app

After this module you can decide what changes between dev, QA, and production, and what stays the same. You can pick the monitoring signal that answers a question about latency, cost, payloads, or callers. You can also pick the security control that blocks a network path, limits a role, or protects a key. The method is simple. Find the deciding constraint in a long question, then reject each wrong option with one reason.



The deciding constraint

Read the last sentence of the question first. It names what the answer must do, such as change, prove, block, or recover. Then scan the middle of the scenario for constraint words such as exact, last approved, no internet, or least operational overhead. Those words remove most options before you read the details.

The tree below starts from the scenario and ends at a service or feature. Use it as a first filter. The later sections explain each branch in more depth.

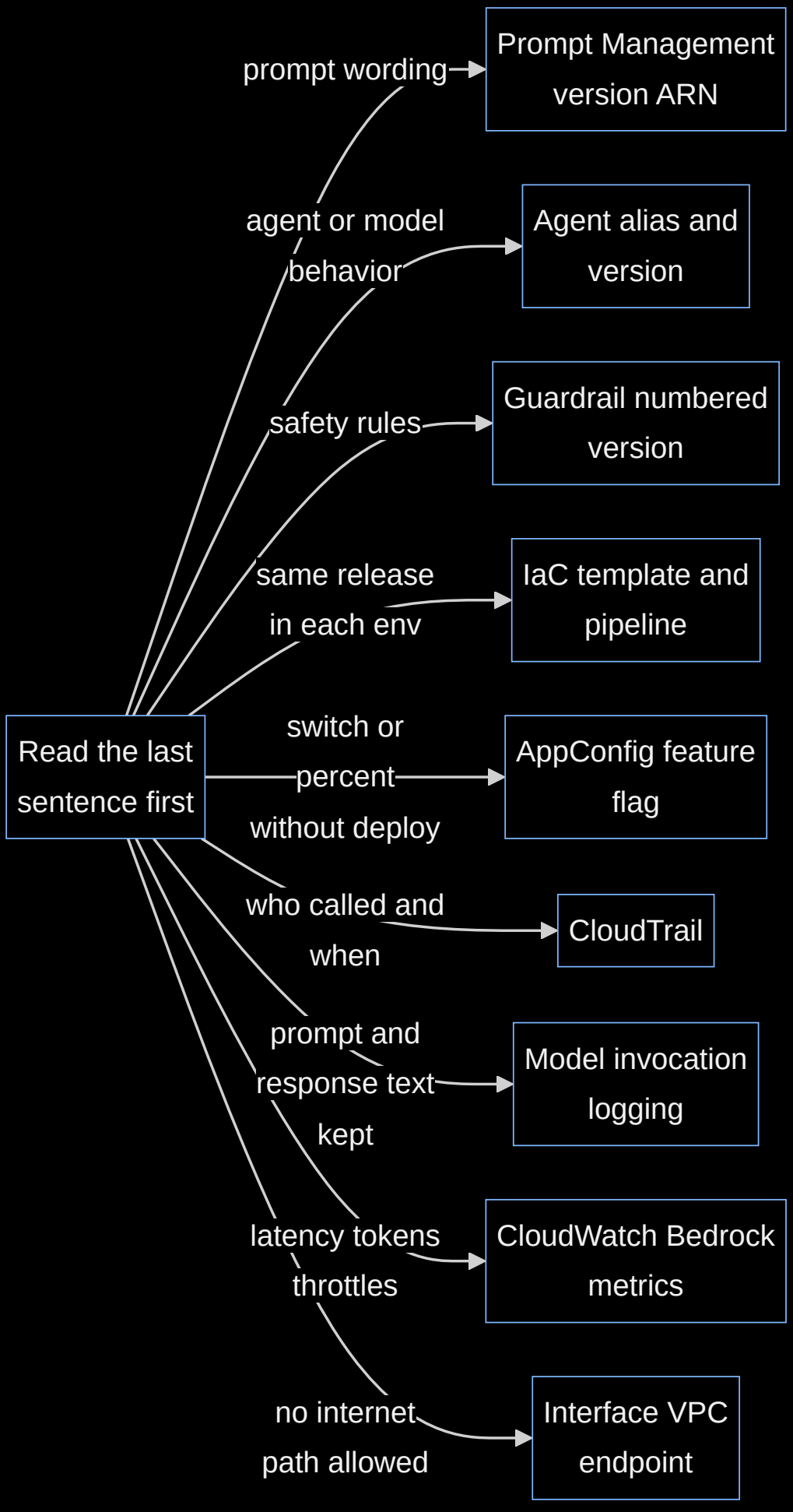


Figure: First filter from the scenario to the service family.

Signal words for each branch:

- Prompt text, template, variables, or reviewed wording points to Prompt Management.
- Agent steps, tool calls, or a release of agent behavior points to agent versions and aliases.
- Blocked topics, PII masking, or harmful content rules point to Bedrock Guardrails.
- Same release in each environment with no drift points to infrastructure as code and a pipeline.
- Flag, percentage, allow list, or log level changed at runtime points to AppConfig.
- Who called and when points to CloudTrail, which is an API audit record.
- Prompt and response text retained for investigation points to model invocation logging.
- Latency, throttles, token counts, or an alarm on a trend points to CloudWatch Bedrock metrics.
- Private subnet with no internet route points to an interface VPC endpoint.

Exam tip: The word exact points to a pinned version, not to the draft. The word last approved points to a stored pointer that you can move back.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring.html and https://docs.aws.amazon.com/bedrock/latest/userguide/logging-using-cloudtrail.html and https://docs.aws.amazon.com/bedrock/latest/userguide/vpc-interface-endpoints.html

---

Environment differences: what changes between QA and production

Most wrong answers in this area mix up two things. One is the artifact that changes, such as a prompt version or a model ID. The other is the place where the change lands, such as an alias, a configuration value, or a separate account. A strong answer names both. A weak answer changes the code.

The matrix below lists each lever. Yes means the environment has its own value or its own control. No means it uses none, or it uses a shared one. The detail list after the matrix explains why each row is set that way.

Lever	Dev	QA	Prod
Own prompt version ARN in config	Yes	Yes	Yes

Lever	Dev	QA	Prod
Own agent alias	Yes	Yes	Yes
Own guardrail version	Yes	Yes	Yes
Own knowledge base and data source	Yes	Yes	Yes
Own model ID or inference profile value	Yes	Yes	Yes
Provisioned Throughput	No	No	Yes
Separate AWS account	Yes	Yes	Yes
Same IaC template with own parameters	Yes	Yes	Yes
Approval gate before this stage	No	No	Yes
Canary or linear traffic shift	No	No	Yes
Feature flag value can differ	Yes	Yes	Yes
Rollback by alias or version	No	Yes	Yes
Quota increase requested for account	No	No	Yes
Verbose logging allowed	Yes	Yes	No
Long log retention	No	No	Yes

Figure: Environment matrix. Yes or No for each lever.

Rule: Build the artifact once. Change only the configuration, the alias, or the version pointer between environments.

Why: a rebuild in each environment creates a different artifact. The tested build and the released build can then differ. A pointer change is small and easy to undo.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/deploy-agent.html and course slide extract pages 376 and 374.

Environment levers in detail

- **Prompt version.** Dev edits the working copy. QA and production reference a fixed prompt version ARN. Production must never read the working copy. Version limits: Not in source.
- **Agent alias.** Dev and QA call a test or staging alias. Production calls a production alias. Moving that alias to a new version is the release. Moving it back is the rollback.
- **Guardrail version.** Dev can use the working copy. QA and production should pin a numbered version. The CloudTrail page records a guardrail version on

ApplyGuardrail events. Version immutability: Not in source.

- **Knowledge base and data source.** Each environment gets its own data source. This stops a QA test from reading production documents. Sync and vector store limits: Not in source.
- **Model ID or inference profile.** Store the value per environment in Parameter Store or AppConfig. Invocation log records name the model ID or inference profile ID that ran.
- **Provisioned Throughput.** Only production buys reserved capacity. Dev and QA use on-demand capacity to test logic. Setup steps and pricing: Not in source.
- **Separate account.** Each environment runs in its own AWS account under one organization. Production gets the strictest service control policies. Those policies do not affect the management account.
- **IaC template.** One template holds the shape of the system. Parameters hold the per-environment values. A CDK app produces a CloudFormation template with cdk synth.
- **Pipeline promotion.** The pipeline passes the same artifact from stage to stage. A manual approval can sit at any stage. The approval gate is the control that blocks an unapproved release.
- **Canary or linear shift.** Use it in production. Lambda aliases and SageMaker endpoints support gradual shifts. ECS services in CodeDeploy use blue green only.
- **Feature flag.** Flag values can differ per environment. A flag change takes effect without a code deployment.
- **Rollback.** QA and production need a fast path back. Repoint an alias, use a stage rollback, or use a CodeDeploy alarm rollback.
- **Quotas.** Quotas belong to each account. Production may need a quota increase in its own account. Bedrock quota values: Not in source.
- **Logging level and retention.** Dev can log more detail for debugging. Production logs what policy allows. It keeps records only as long as policy needs them.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/deploy-agent.html and https://docs.aws.amazon.com/bedrock/latest/userguide/logging-using-cloudtrail.html and https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html and https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html and course slide extract pages 10, 314, 376, 377, 383, 388, 389, 577, and 364.

Decision tree: which lever changes

Start from the artifact that changed. The lever is the pointer to that artifact in the environment. Do not change code when a pointer is enough.

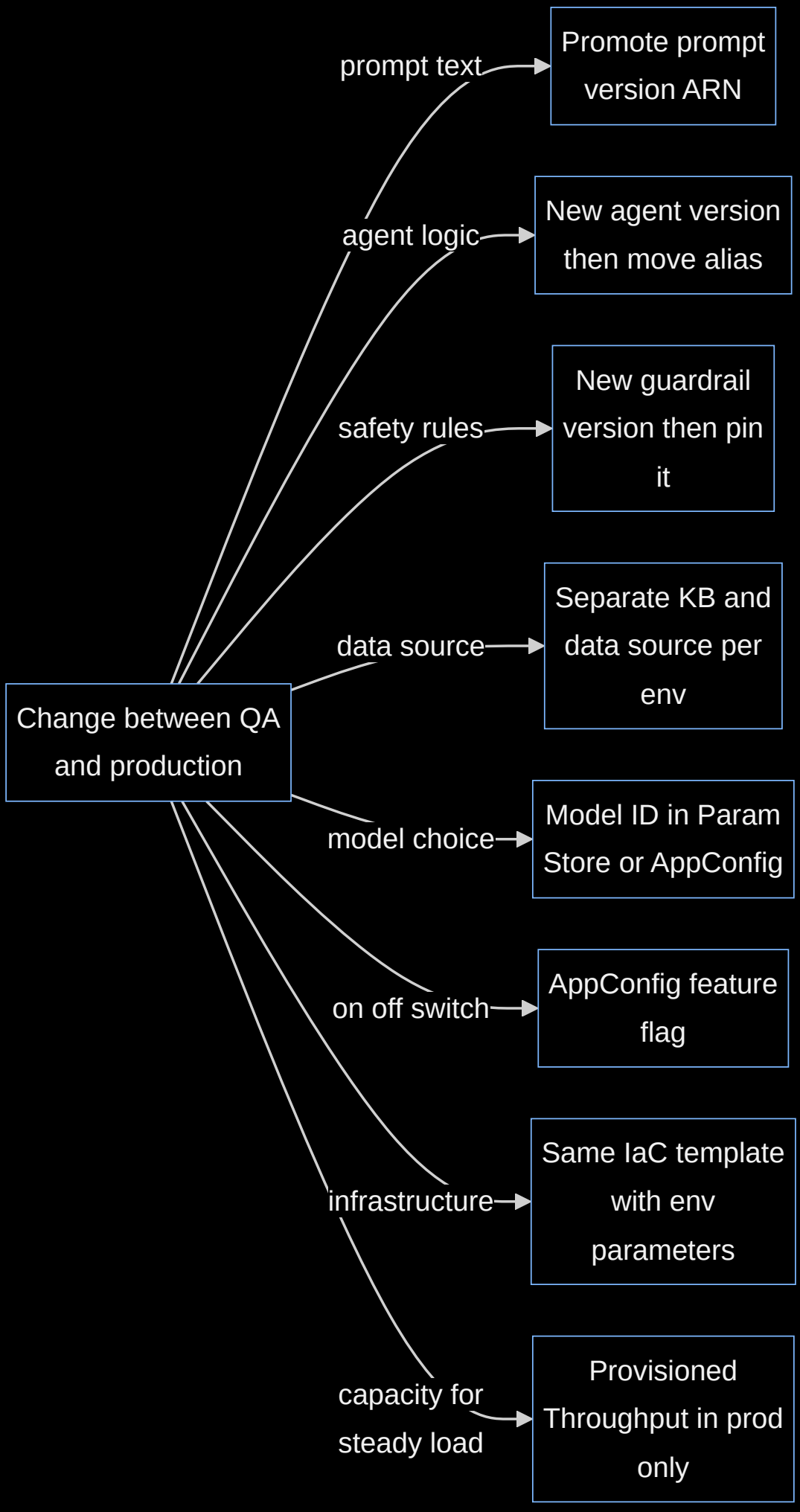


Figure: Each artifact type maps to one environment lever.

Rule: If the change is text, use a version. If the change is a switch, use a flag. If the change is a new resource, use the template.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/deploy-agent.html and https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html and course slide extract pages 364 and 577.

Decision tree: how a release moves to production

The release path has one artifact, one gate per environment, and one controlled shift to live traffic. A failed gate stops the stage. A bad live signal sends traffic back.

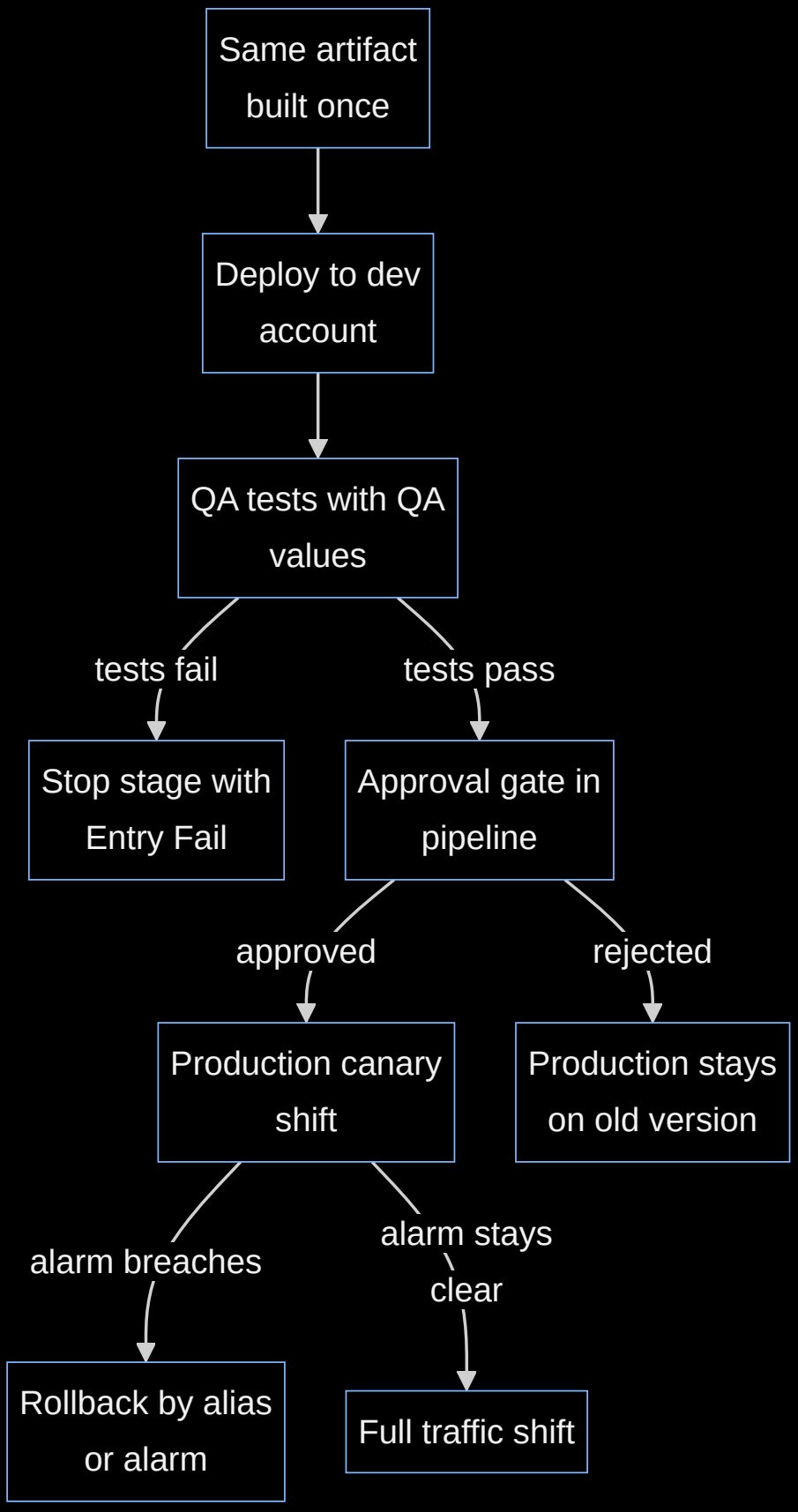


Figure: Promotion path with two gates and one rollback branch.

Why: each gate answers a different question. The QA test asks whether the artifact works. The approval asks whether a person accepts the risk. The alarm asks whether live traffic behaves.

Source: https://docs.aws.amazon.com/codepipeline/latest/userguide/stage-conditions.html and course slide extract pages 374, 383, and 388.

Decision tree: how a bad release goes back

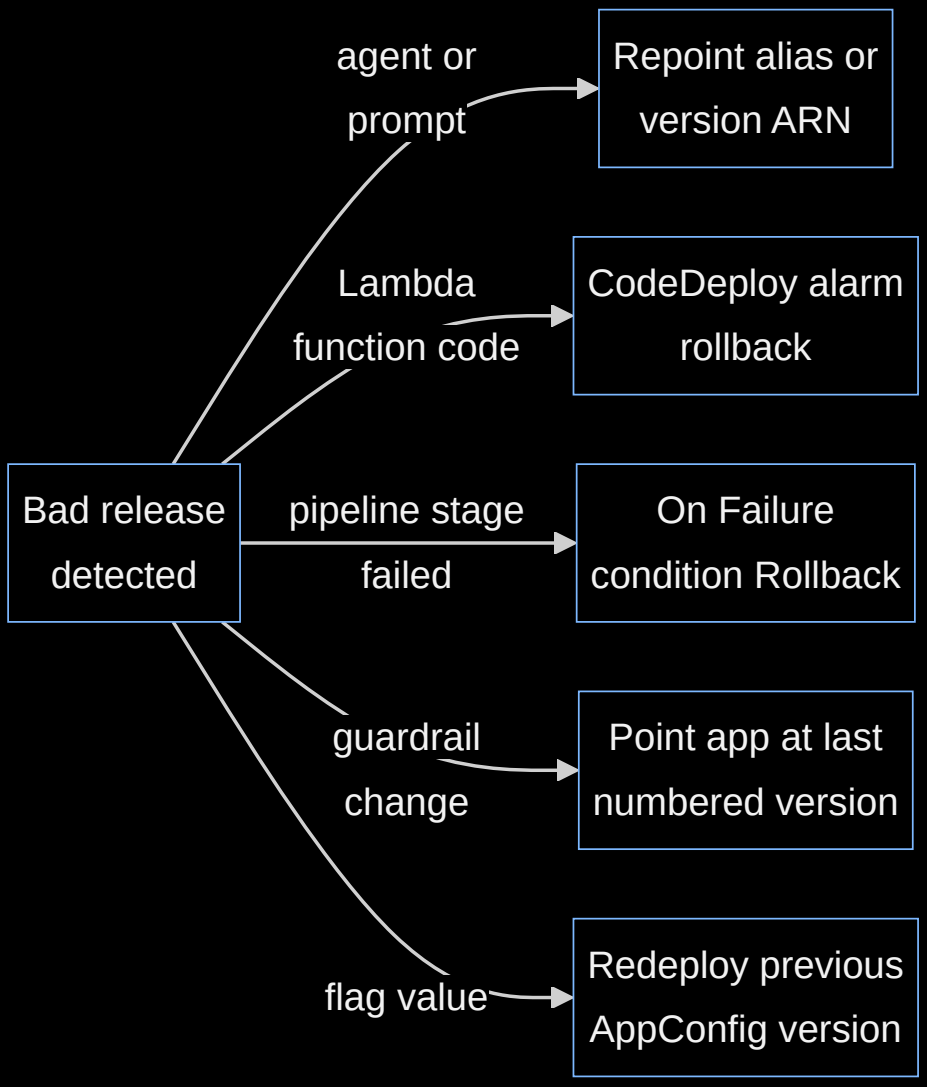


Figure: Rollback path depends on what changed.

Rule: Roll back by moving a pointer that already exists. Do not rebuild a previous version under pressure.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/deploy-agent.html and https://docs.aws.amazon.com/codepipeline/latest/userguide/stage-conditions.html and course slide extract page 383.

Decision tree: which model identifier

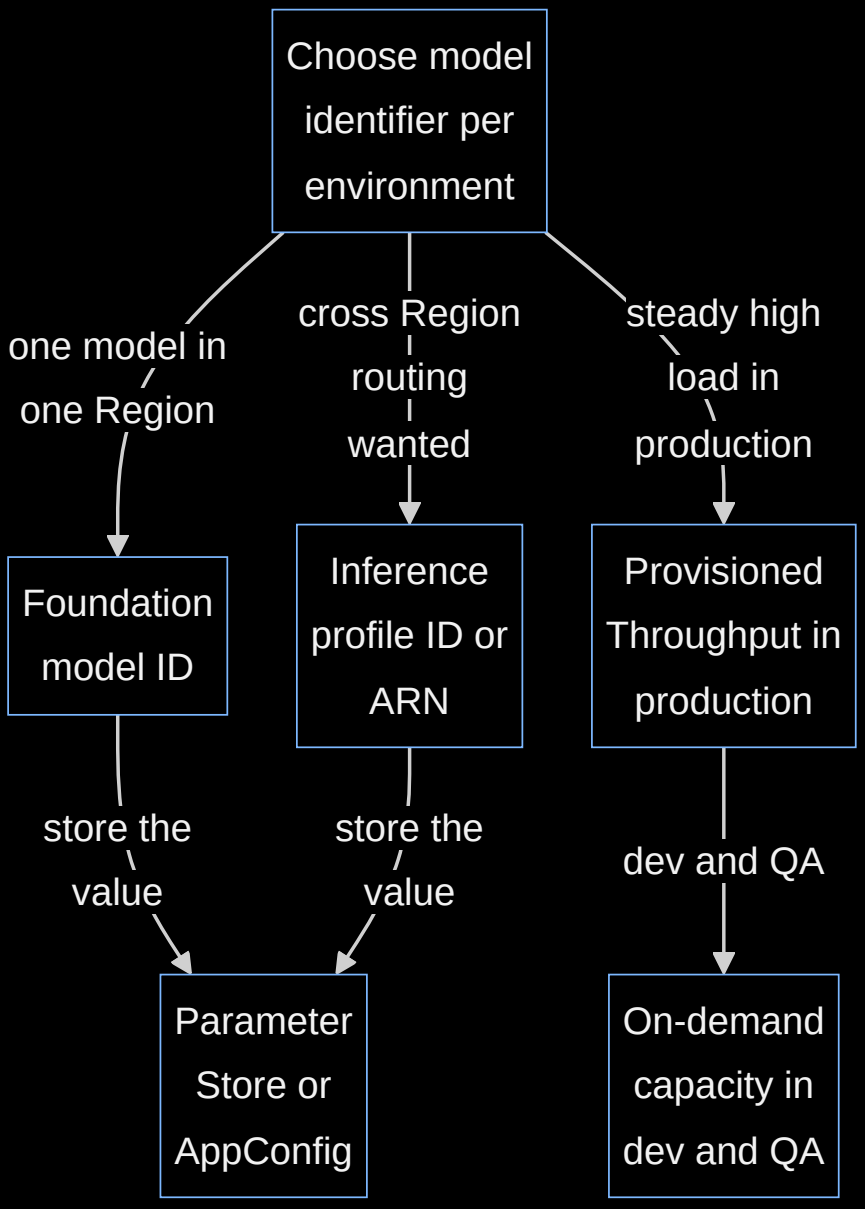


Figure: Model identifier choice and where the value lives.

Fact: The model invocation log record holds a field for the model ID or inference profile ID used for the call.

Trap: Hard-coding one model ID in each service makes a QA model change a code release.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html and course slide extract page 619. Provisioned Throughput details: Not in source.

Services and features in scope

Each section below covers one service or feature. It states the fact first, then the rule for when to choose it, then the trap for when not to choose it. Numbers that the fetched sources do not confirm are marked Not in source.

Amazon Bedrock Prompt Management

Fact: Prompt Management stores reusable prompt templates. Templates take variables at runtime. The course slide places it in the governance group with traces and evaluations. The application refers to a managed prompt by its ARN.

Rule: Use it when several applications share prompt text and each change needs review before production.

Fact: CloudTrail logs RenderPrompt as a data event on the prompt resource type. RenderPrompt is a permission-only action. It renders a managed prompt for InvokeModel or Converse.

Trap: Keeping prompt text in code or in an environment variable leaves no managed version record for an auditor.

Why: a version is the unit that QA tests and that production uses. Once versions exist, the question becomes which version each environment points to.

Limits and numbers: version count, template size, and version immutability are Not in source. The Prompt Management documentation page did not return text in this session.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/logging-using-cloudtrail.html and course slide extract page 10.

Bedrock Agents versions and aliases

Fact: The working draft changes when you edit the agent. A version is a numbered snapshot that starts at 1. A version is immutable. An alias points to one version, and the application calls the alias.

Rule: Call a production alias from production code. Move the alias to a new version to release. Move it back to roll back.

Fact: Setting aliasInvocationState to REJECT_INVOCATIONS pauses an alias. Setting it to ACCEPT_INVOCATIONS resumes it. No IAM change is needed to pause.

Trap: Calling the working draft in production sends unreleased edits to users.

Trap: The documentation banner says Bedrock Agents Classic is not open to new customers. A new design should check the current agent offering first.

Why: the alias gives the application one stable name. The version behind it can change without an application change.

Limits and numbers: alias count and version count: Not in source.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/deploy-agent.html

Bedrock Guardrails versions

Fact: A guardrail has a working draft and numbered versions. A call or an ApplyGuardrail request names a guardrail and a version. The CloudTrail page shows a guardrailVersion request parameter on ApplyGuardrail data events.

Rule (design choice): Test the working draft in QA. Pin a numbered version in production so a later edit cannot change production behavior.

Fact: When an organization enforces a guardrail and the request names none, the event shows ENFORCED as a marker in the guardrail fields. The marker is not an identifier.

Trap: Pointing production at the working draft lets an edit change safety behavior with no release record.

Limits and numbers: version immutability, version count, and enforcement set-up steps are Not in source. The guardrail versioning page did not return text in this session.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/logging-using-cloudtrail.html

Knowledge bases and data sources

Fact: A knowledge base reads from a data source. In this design the data source is an S3 location. CloudTrail data events exist for Retrieve and RetrieveAndGenerate on the knowledge base resource type.

Rule: Use a separate knowledge base or at least a separate data source per environment. QA must never index production documents.

Trap: One shared knowledge base across QA and production lets a QA test read real customer documents.

Why: a retrieval answer is only as safe as the documents behind it. Separate sources keep test data and live data apart.

Limits and numbers: sync behavior, vector store limits, and document count limits are Not in source. The knowledge base documentation was not fetched.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/logging-using-cloudtrail.html

AWS Systems Manager Parameter Store

Fact: Parameter Store is a serverless store for configuration and secrets. Values can be plain text or encrypted with KMS. It tracks versions, and IAM controls access to each parameter.

Rule: Store per-environment values here, such as a model ID, a guardrail version number, or an alias name. The app reads them at startup.

Trap: Parameter Store alone does not split traffic or roll back on an alarm. Use AppConfig for a gradual change.

Why: a plain setting that changes rarely does not need the extra machinery of a feature flag service.

Limits and numbers: throughput and parameter size limits: Not in source.

Source: course slide extract page 619.

AWS Secrets Manager

Fact: Secrets Manager stores secrets encrypted with KMS. It can force rotation on a set schedule. It can run a Lambda function to generate and rotate a secret. It can replicate a secret to another Region. The course slide says it is mostly used for database integration.

Rule: Use it for a database password or an API key that must rotate without a code change.

Trap: Putting a secret in code, a plain environment variable, or a configuration file breaks the rule that secrets stay encrypted and out of code.

Limits and numbers: price per secret and API call rates: Not in source.

Source: course slide extract pages 495, 500, and 501.

AWS AppConfig

Fact: AppConfig deploys dynamic configuration to running applications without a code deployment. It supports feature flags and free-form configuration. It can validate a change with a JSON Schema or a Lambda validator before deployment. It rolls back automatically when a CloudWatch alarm fires.

Fact: The configuration can come from the AppConfig hosted store, Secrets Manager, Parameter Store, or S3. The AppConfig Agent caches the deployed configuration for the app.

Rule: Use it for flags, allow lists, throttles, and log levels that must change at runtime and roll back fast.

Trap: AppConfig tracks configuration deployments. It does not give prompt-level review history. Use Prompt Management for prompt versions.

Why: a runtime change is the highest-risk change in a GenAI app. Validation and an alarm-based rollback reduce the blast radius without a release.

Limits and numbers: named deployment strategies, validator limits, and retrieval call rates: Not in source for named strategies.

Source: https://docs.aws.amazon.com/appconfig/latest/userguide/what-is-appconfig.html and course slide extract page 364.

AWS CodePipeline stage conditions

Fact: An Entry condition checks run ahead of a stage. Its results are Fail or Skip. An On Failure condition runs when a stage fails. Its only result is Rollback. An On Success condition runs after a stage succeeds. Its results are Rollback or Fail.

Fact: Rules include a CloudWatch alarm check, a Lambda check, a variable check, and a deployment window check. A Skip result cannot be overridden. Automatic stage retry cannot combine with On Failure.

Rule: Use an Entry condition with Fail to stop a stage from starting when a gate is not met. Use On Failure with Rollback to restore the last good state after a failed deployment.

Rule: Use On Success with a check when a finished deployment must pass a test before the release is kept.

Trap: On Failure does not accept Fail as a result. A failed audit that must stop entry needs an Entry condition.

Trap: A Skip result ends the stage quietly. It does not show a failed gate as a failure.

Limits and numbers: a rollback can target only a previous execution in the current pipeline structure version.

Source: https://docs.aws.amazon.com/codepipeline/latest/userguide/stage-conditions.html and course slide extract pages 374 and 377.

AWS CodeDeploy and traffic shifting

Fact: CodeDeploy shifts Lambda alias traffic with canary and linear options. On ECS, CodeDeploy uses blue green only. SageMaker endpoint deployment guardrails support blue green, canary, linear, and all-at-once shifts, with automatic rollback.

Fact: CodeDeploy rolls back automatically when a deployment fails or when a CloudWatch alarm you attach fires.

Rule: Choose canary to send a small first share of traffic, then the rest. Choose linear for even steps over time. Attach an alarm on errors and latency.

Trap: Selecting linear or canary for ECS in CodeDeploy is wrong. ECS allows blue green only.

Limits and numbers: step percentages and minutes are shown as examples on the slide. Treat the exact names as Not in source for this module.

Source: course slide extract pages 314, 383, 388, and 389.

CloudFormation, CDK, and StackSets

Fact: CloudFormation is declarative. A template names the resources, and CloudFormation creates them in the right order. CDK writes infrastructure in a programming language. cdk synth turns the app into CloudFormation templates. Both CDK and SAM use CloudFormation underneath.

Rule: Keep one template for the shape of the system. Pass per-environment values as parameters. The pipeline deploys the same template to each account.

Trap: Copying a template into each account by hand creates drift. Two accounts then run two different systems.

StackSets fact: a StackSet pushes one template out to many accounts and Regions. Service-managed permissions and automatic deployment for new OU accounts: Not in source. The StackSet documentation was not fetched.

Source: course slide extract pages 577, 578, and 585.

AWS Organizations and service control policies

Fact: A service control policy sets the maximum permissions for IAM users and roles in member accounts. An SCP never grants a permission. An SCP does not affect the management account. It applies to the root user of a member account.

Fact: The effective permission is the overlap of the SCP, any permissions boundary, and the identity policy. An explicit deny in an SCP blocks the action in that account.

Rule: Use an SCP on an OU to stop production accounts from using a service or action that policy forbids.

Trap: An SCP alone never gives Bedrock access to anyone. The identity policy must still allow the call.

Trap: Attaching an SCP at the root reaches every member account at once. Test on one OU first.

Limits and numbers: SCP size limit and attachment limits: Not in source for values.

Source: https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html and course slide extract page 489.

Provisioned Throughput

Fact: Provisioned Throughput reserves model capacity for a steady load. Setup steps, unit size, commitment terms, and pricing: Not in source. The fetched pages did not cover it.

Rule (design choice): Buy reserved capacity only in production, and only when traffic is steady. Use on-demand in dev and QA.

Trap: Reserved capacity in QA adds fixed cost. It also gives QA no more information about the release logic than on-demand does.

Why: the production load pattern is what justifies a fixed cost. QA needs correct behavior, not a reserved rate.

Source: Not in source for Provisioned Throughput details. The environment rule comes from the module design.

CloudWatch metrics, dashboards, and alarms for Bedrock

Fact: Bedrock runtime metrics are in the AWS/Bedrock namespace. They include Invocations, InvocationLatency, InvocationClientErrors, InvocationServerErrors, InvocationThrottles, InputTokenCount, OutputTokenCount, and TimeToFirstToken.

Fact: Invocations counts accepted requests. Throttled calls and calls refused at input checks are left out of it. Throttled requests appear only in InvocationThrottles. Requests that fail before the model is known have no ModelId dimension.

Fact: TimeToFirstToken applies only to ConverseStream and InvokeModelWithResponseStream. EstimatedTPMQuotaUsage is an approximation. The docs say not to use it as the only capacity signal.

Fact: Output tokens per second is OutputTokenCount divided by the difference of InvocationLatency and TimeToFirstToken, times 1000. It applies only to streaming operations. The docs example alarms on the p50 statistic over 5-minute periods, with 3 of 5 datapoints breaching.

Rule: Alarm on server errors and throttles in production. Query both with and without the ModelId dimension so no failed request is missed.

Trap: Invocations is not an audit count. Throttled calls are missing from it.

Trap: OTPS for a nonstreaming call is not valid, because TimeToFirstToken is missing.

Limits and numbers: a CloudWatch dimension limit of 30 per metric is shown on the course slide. Dashboard and alarm quotas for Bedrock: Not in source.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring-runtime-metrics.html and https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring-runtime-otps.html and https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring.html and course slide extract page 109.

Model invocation logging

Fact: Model invocation logging is disabled by default. It is set per account and per Region. Supported destinations are CloudWatch Logs and S3. Both belong to the account and Region that hold the configuration.

Fact: It covers Converse, ConverseStream, InvokeModel, and InvokeModelWithResponseStream on the bedrock-runtime endpoint. JSON request and response bodies of 100 KB or less stay inline in the log. Larger bodies and binary data go to S3 as separate objects.

Fact: requestMetadata is the only field the caller supplies. The record also holds the caller identity ARN, the model ID or inference profile ID, and token counts. Logs stay until the logging configuration is deleted.

Rule: Enable it in the account and Region where prompt text must be kept. Set retention on the CloudWatch log group or an S3 lifecycle rule, because the docs say logs stay until the configuration is deleted.

Trap: A central logging account in another account is not a supported destination for these logs.

Trap: The four operations listed do not include InvokeAgent. Agent calls need CloudTrail data events and agent tracing instead.

Trap: Calls through the bedrock-mantle endpoint are not captured by this logging.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html and https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring.html

AWS CloudTrail

Fact: CloudTrail records API calls made by users, roles, and services. Event history covers recent events by default. A trail delivers an ongoing record to S3, and a trail applies to all Regions by default.

Fact: Converse, ConverseStream, InvokeModel, and InvokeModelWithResponseStream are management events. InvokeAgent, RenderPrompt, ApplyGuardrail, and knowledge base Retrieve calls are data events. Data events are off by default and need advanced event selectors.

Fact: A streaming call also writes a stream-completion event with eventType AwsServiceEvent. It holds token counts. It is not an API call and IAM and SCP do not evaluate it. Filter eventType to AwsApiCall for counts and authorization review.

Rule: Use CloudTrail to answer who called an API and when. Turn on data events for agent, prompt, and guardrail calls when the audit needs them.

Trap: CloudTrail data events are off by default. A default trail shows no InvokeAgent calls.

Trap: Counting stream-completion events as well as API calls doubles the count.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/logging-using-cloudtrail.html and course slide extract pages 448, 449, 450, and 451.

AWS X-Ray

Fact: X-Ray traces one request across services. The application SDK captures calls to AWS services and HTTP calls. The X-Ray daemon sends the traces. Lambda can enable active tracing. The role needs permission to write traces.

Rule: Use X-Ray when the question asks where time went across services for one request.

Trap: X-Ray shows timing and call paths. It is not the record of prompt and response text. Use invocation logs for the text.

Limits and numbers: sampling rate defaults and trace retention: Not in source.

Source: course slide extract pages 454, 459, and 461.

AgentCore Observability

Fact: AgentCore is the newer agent platform named by the Bedrock Agents Classic banner. Its observability signals, dashboards, and trace formats: Not in source. The AgentCore documentation was not fetched for this module.

Rule (design choice): For a new agent design, check the AgentCore observability pages before you pick a trace or log design.

Trap: Do not claim a specific AgentCore metric or log name without a source. The module does not verify those names.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/deploy-agent.html for the banner only. Observability details: Not in source.

IAM least privilege and condition keys

Fact: IAM policies are JSON documents attached to users, groups, and roles. Least privilege means granting only the actions and resources a principal needs. Roles for services let AWS services act on your behalf.

Rule: Give the application role only the Bedrock actions and the resource ARNs it uses, such as one prompt version or one alias.

Fact: RenderPrompt is permission-only. A role that renders managed prompts needs that action in its policy, even though the application does not call it directly.

Trap: A role with a wildcard resource lets the application call any model in the account.

Limits and numbers: the exact condition key name for guardrails, and the full list of Bedrock condition keys: Not in source. The service authorization reference did not return text.

Source: course slide extract pages 477 and 484, and https://docs.aws.amazon.com/bedrock/latest/userguide/logging-using-cloudtrail.html

Resource policies and endpoint policies

Fact: An endpoint policy on an interface endpoint names principals, actions, and resources allowed through that endpoint. The default endpoint policy allows full access. A custom policy narrows it. A resource-based policy grants access on the resource itself.

Rule: Use an endpoint policy to limit which role may use the private path, and keep the identity policy narrow too.

Trap: An endpoint policy does not replace the IAM policy. The call must be allowed by both, because access requires each layer to allow it.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/vpc-interface-endpoints.html and https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html

AWS KMS customer managed keys

Fact: KMS keys are integrated with IAM and CloudTrail. AWS owned keys and AWS managed keys are free. A customer managed key has a monthly fee on the course slide, and automatic rotation must be enabled for it. The slide shows a per-call fee. Current prices: Not in source.

Fact: For S3 SSE-KMS, the upload header names KMS, and each object calls KMS. KMS request quotas apply per second by Region, and the slide notes a quota increase is requested through Service Quotas.

Fact: Model invocation logs to S3 with SSE-KMS need a key policy that lets the Bedrock service principal call GenerateDataKey, under the account and ARN conditions shown in the docs.

Rule: Use a customer managed key when the team must control the key policy and rotation for audit evidence.

Trap: A default AWS owned or AWS managed key does not meet a rule that needs your own key policy.

Source: course slide extract pages 210, 211, 495, 496, and 497, and https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html

VPC endpoints and PrivateLink for Bedrock

Fact: Bedrock uses interface VPC endpoints powered by PrivateLink. The service names include bedrock, bedrock-runtime, bedrock-agent, and bedrock-agent-runtime. Private DNS lets the SDK keep the default Regional hostname.

Fact: Without private DNS, the SDK must set the endpoint URL to the VPC endpoint address. Interface endpoints do not need an internet gateway, NAT device, VPN, or Direct Connect. Instances do not need public IP addresses.

Fact: Gateway endpoints exist for S3 and DynamoDB. The course slide says interface endpoints serve most other services. The Bedrock page lists only interface endpoints.

Rule: Use an interface endpoint for bedrock-runtime from private subnets. Add bedrock-agent-runtime when the app calls agents. Enable private DNS to keep standard hostnames.

Trap: A gateway endpoint is not offered for Bedrock in the fetched source. Gateway endpoints cover S3 and DynamoDB.

Trap: A NAT gateway gives private subnets outbound internet access. It fails a rule that forbids any internet path.

Trap: Turning off private DNS keeps the traffic private, but it forces endpoint URLs into code and breaks the hostname rule.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/vpc-interface-endpoints.html and course slide extract pages 518, 520, and 522.

Cross-account access

Fact: IAM Identity Center can manage permission sets across accounts in an organization. An SCP does not directly affect resource-based policies, and it does not restrict principals outside the organization.

Fact: Destinations for these logs sit only in the configuring account and Region. CloudWatch cross-account observability: Not in source.

Rule: Keep invocation logs in the account that makes the calls, then share a read view if a central team needs it.

Trap: A bucket in another account cannot receive these invocation logs from this account.

Trap: An SCP in the workload account does not block a bucket policy in a partner account.

Source: course slide extract page 489, https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html, and https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html

Data not used for training

Fact: The course slide that covers security and governance says that the product does not train models on customer data. That slide describes a business intelligence product, not Bedrock.

Rule: For a training-use answer on Bedrock, cite the service data terms, not the slide. Bedrock-specific statement: Not in source.

Trap: Do not turn a claim about one product into a claim about another product.

Source: course slide extract page 281. Bedrock data terms: Not in source.

Confusable pairs: X versus Y

Each pair below has one decision tree, a three-row comparison, one question to ask, and one trap. Use the question first. It usually settles the pair in a few seconds.

Parameter Store versus Secrets Manager

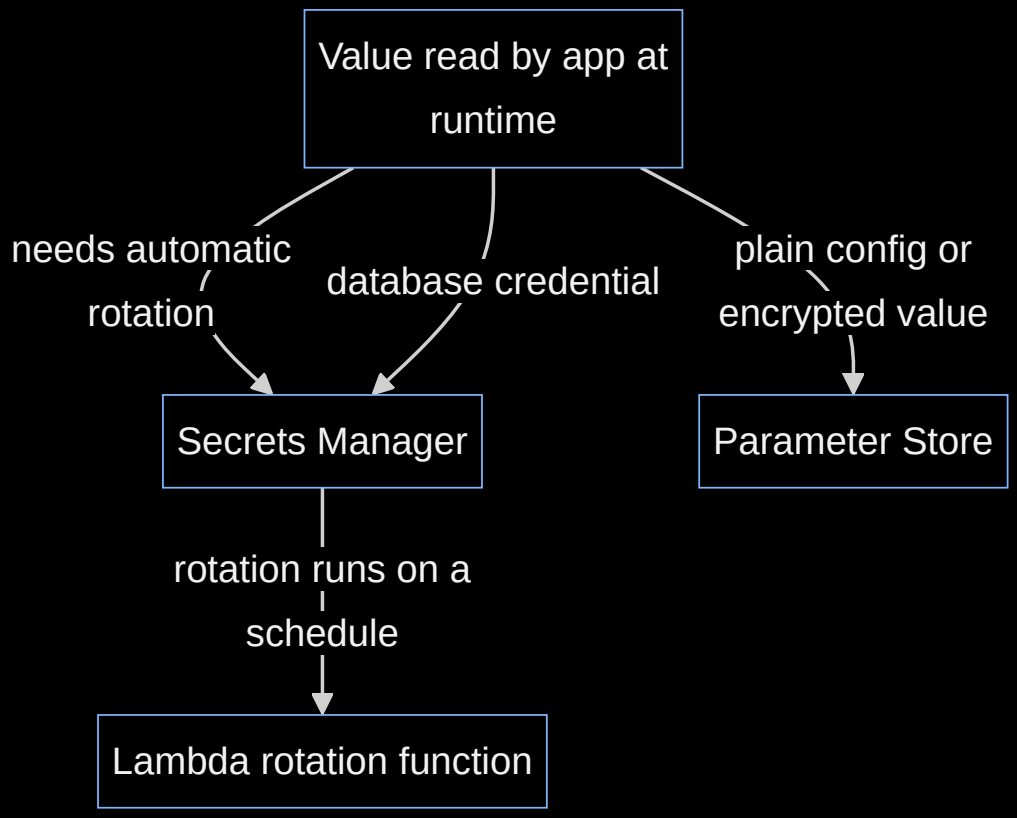


Figure: Rotation need decides the store.

- Purpose: Parameter Store holds settings and optional secrets. Secrets Manager holds secrets that rotate and replicate.
- Input: Parameter Store takes a name and a value, with optional KMS encryption. Secrets Manager takes a secret with a rotation schedule.
- Output or effect: Parameter Store returns the stored value. Secrets Manager returns the current secret and rotates it on schedule.

Question to ask yourself: Does this value need automatic rotation?

Trap: Choosing Parameter Store for a database password that must rotate moves the rotation work to the team.

Source: course slide extract pages 500, 501, and 619.

AppConfig versus Parameter Store

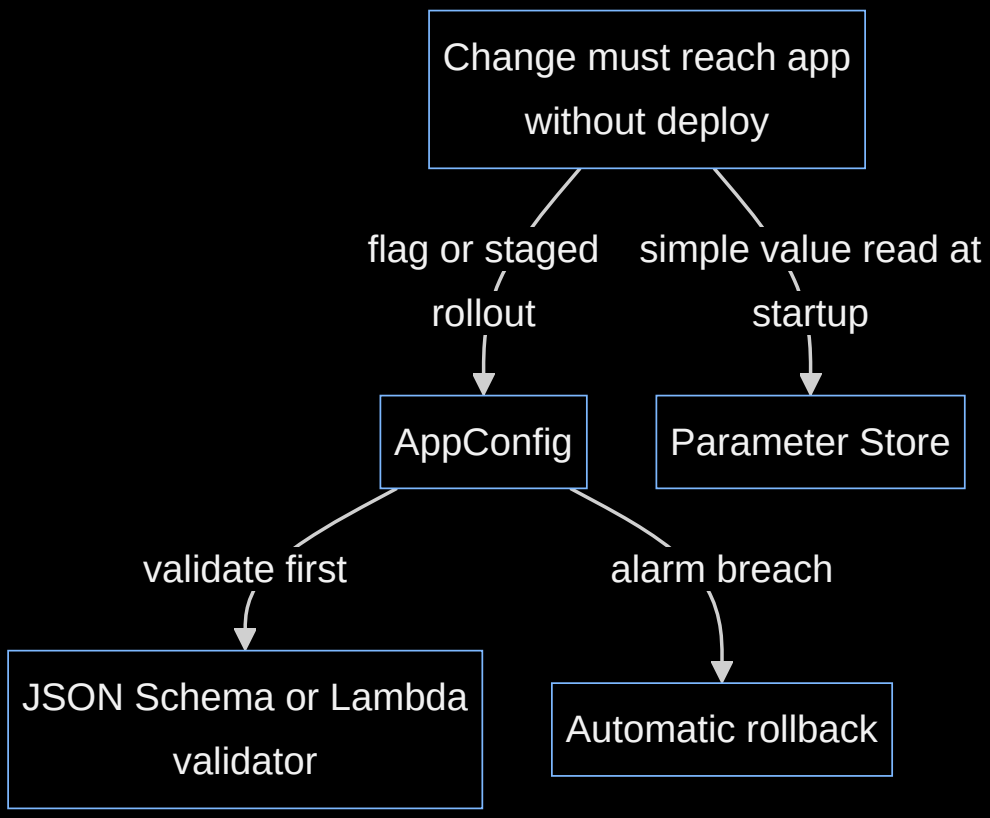


Figure: Rollout and rollback needs point to AppConfig.

- Purpose: AppConfig deploys runtime changes with validation and a monitored rollout. Parameter Store stores values that apps read.
- Input: AppConfig takes a configuration profile, validators, and a deployment. Parameter Store takes a parameter name.
- Output or effect: AppConfig serves a deployed version through its agent and rolls back on alarm. Parameter Store returns the current value on each read.

Question to ask yourself: Must the change roll out in steps and roll back on an alarm?

Trap: Reading Parameter Store on every request for a flag adds calls. The AppConfig Agent caches the deployed configuration for the app.

Source: https://docs.aws.amazon.com/appconfig/latest/userguide/what-is-appconfig.html and course slide extract page 619.

CodePipeline stage conditions versus CodeDeploy alarm rollback

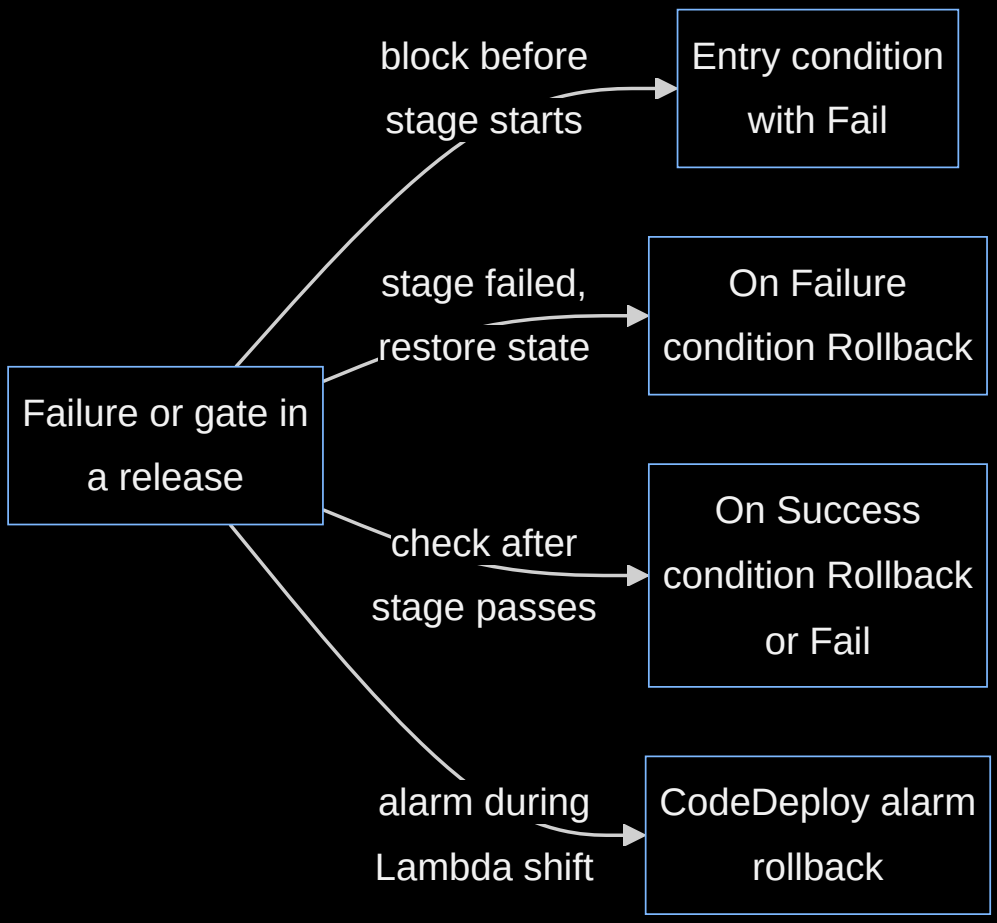


Figure: Stage conditions gate a stage. CodeDeploy rolls back one deployment.

- Purpose: Stage conditions gate or react to a whole pipeline stage. CodeDeploy rolls traffic back for one deployment.
- Input: Stage conditions take rules such as an alarm check or a Lambda check. CodeDeploy takes a deployment group with alarms.
- Output or effect: A stage condition blocks entry, rolls back a stage, or fails a stage. CodeDeploy redeploys the last good revision on an alarm.

Question to ask yourself: Is the check about a stage in the pipeline, or about traffic on one function?

Trap: On Failure accepts only Rollback. A failed audit that must stop entry needs an Entry condition with Fail.

Source: https://docs.aws.amazon.com/codepipeline/latest/userguide/stage-conditions.html and course slide extract page 383.

Interface endpoint versus gateway endpoint versus NAT gateway

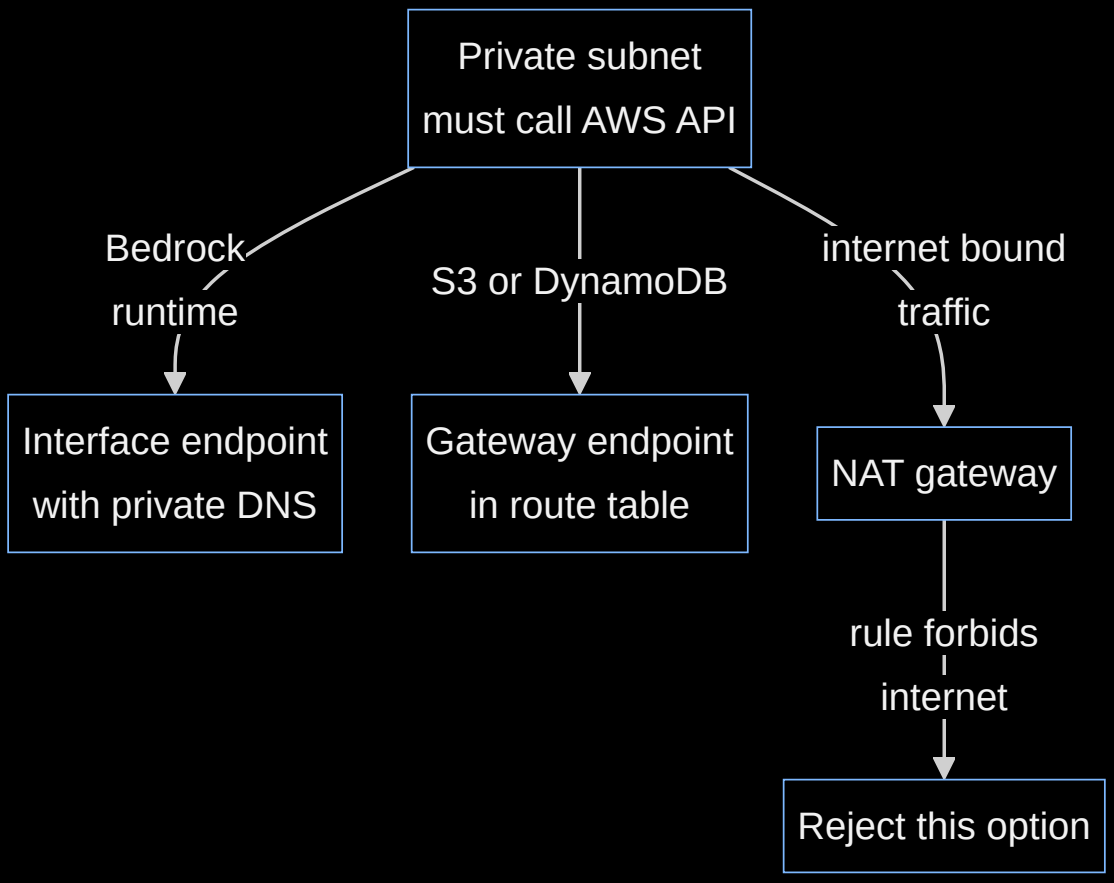


Figure: The private path and the forbidden path.

- Purpose: An interface endpoint gives a private entry point for Bedrock APIs. A gateway endpoint gives private routes for S3 and DynamoDB. A NAT gateway gives private subnets outbound internet access.
- Input: An interface endpoint takes subnets and security groups, with optional private DNS. A gateway endpoint takes route table entries.
- Output or effect: An interface endpoint keeps Bedrock traffic on the AWS network. A NAT gateway sends traffic to the internet.

Question to ask yourself: Does the rule forbid any path to the public internet?

Trap: A NAT gateway looks private because the instances have no public IP addresses. The NAT gateway still reaches the internet.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/vpc-interface-endpoints.html and course slide extract pages 518 and 520.

CloudTrail versus model invocation logging

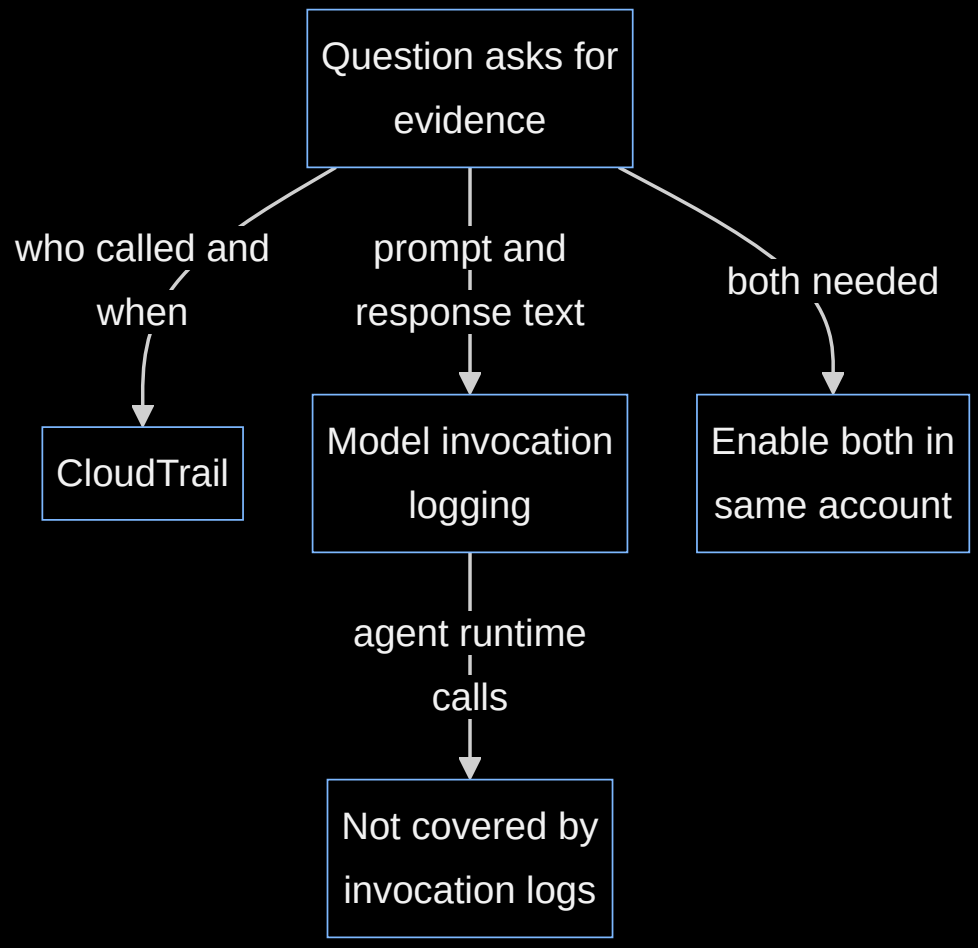


Figure: Caller evidence and content evidence are separate records.

- Purpose: CloudTrail records API calls for audit. Invocation logging records model input and output for supported runtime calls.
- Input: CloudTrail takes management and data events from the account. Invocation logging takes four runtime operations on bedrock-runtime.
- Output or effect: CloudTrail gives the caller identity, the time, and the API name. Invocation logging gives bodies up to the size limit, token counts, and metadata.

Question to ask yourself: Does the auditor need the words sent and returned, or only the caller and the time?

Trap: Invocation logging does not cover InvokeAgent. For agent calls, use CloudTrail data events and the agent tracing setting.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/logging-using-cloudtrail.html and https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html

CloudWatch metrics versus X-Ray

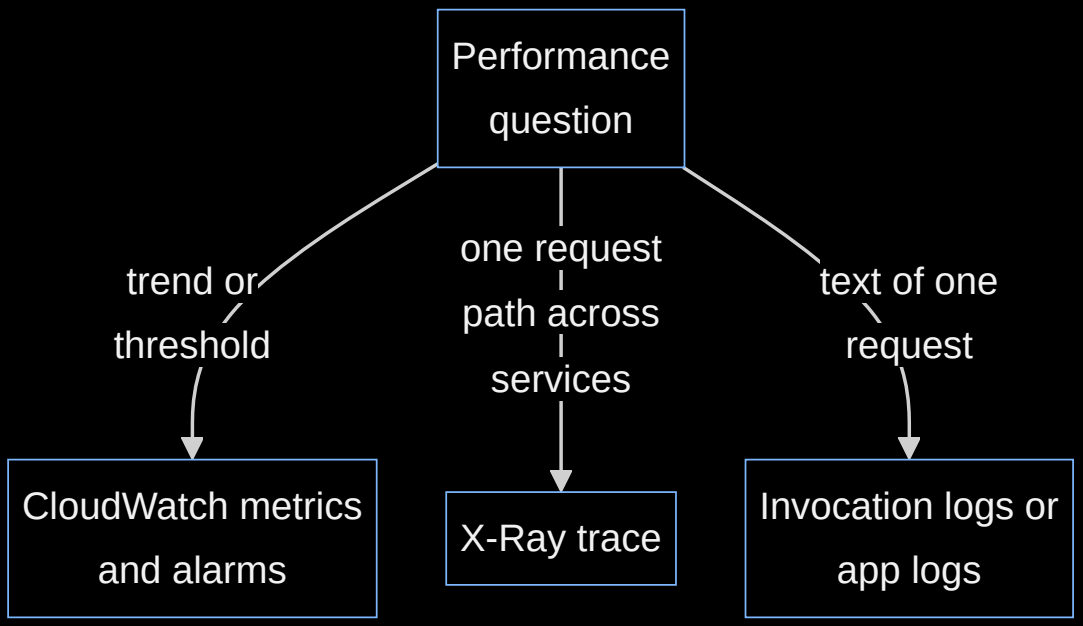


Figure: Trend, path, and text answer different questions.

- Purpose: Metrics show volume, latency, and errors over time. X-Ray shows the path of one request across services.
- Input: Metrics take the AWS/Bedrock namespace and dimensions such as ModelId. X-Ray takes traces from an instrumented SDK.
- Output or effect: Metrics drive alarms and dashboards. Traces show which call took the time.

Question to ask yourself: Is the question about a trend across many calls, or about one slow request?

Trap: A metric is not a per-request record. A dashboard cannot show the prompt of one failed call.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring-runtime-metrics.html and course slide extract pages 109, 459, and 461.

Service control policy versus IAM policy versus permission boundary

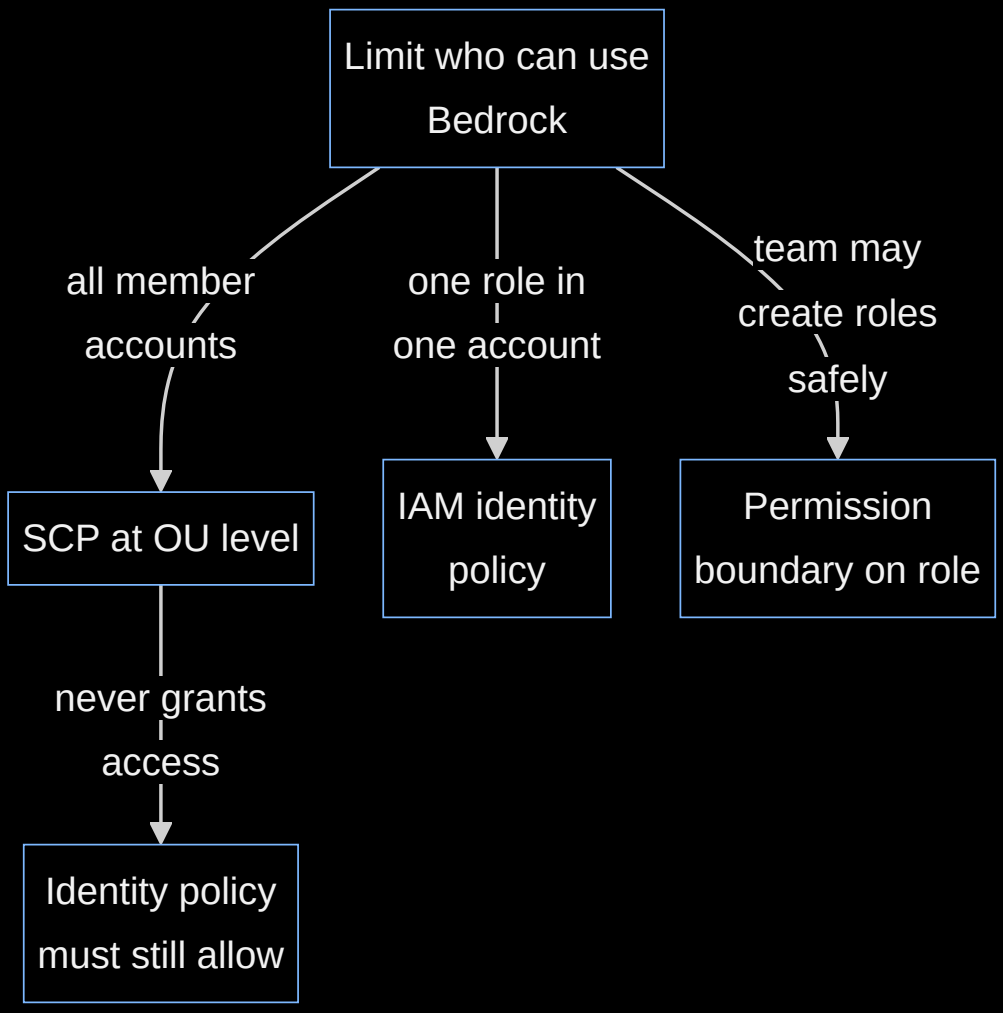


Figure: Caps and grants are separate layers.

- Purpose: An SCP caps permissions for member accounts. An IAM policy grants permissions to a principal. A permission boundary caps what one role can have.
- Input: An SCP attaches to the root, an OU, or an account. An IAM policy attaches to a user, group, or role.
- Output or effect: An SCP never grants. A call works only when the SCP, any boundary, and the identity policy all allow it.

Question to ask yourself: Is the control across many accounts, or inside one role?

Trap: An SCP alone never lets a user call Bedrock. The identity policy must allow the call too.

Source: https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html and course slide extract page 477.

Prompt Management versus prompt files in S3

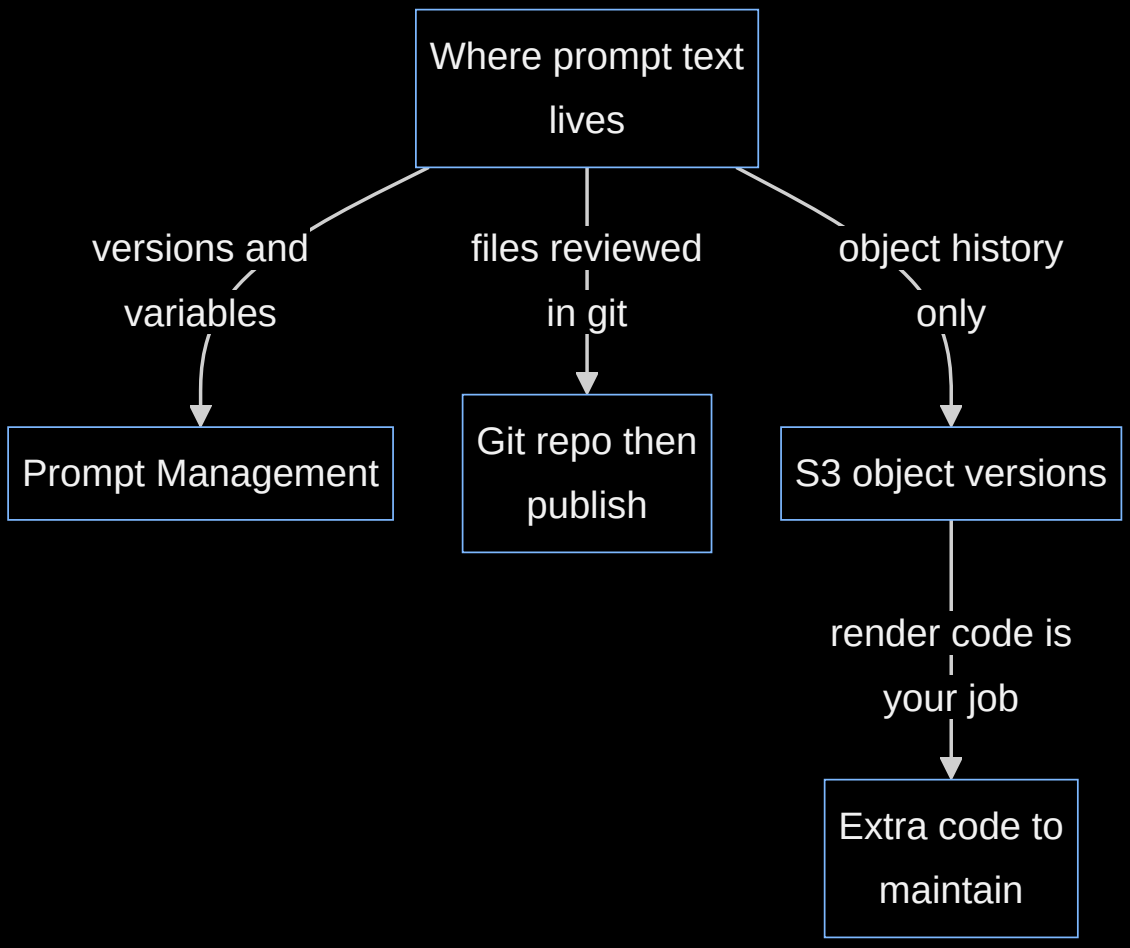


Figure: Managed rendering versus owned rendering code.

- Purpose: Prompt Management stores templates as managed resources with variables. S3 stores files with object history.
- Input: Prompt Management takes template text, variables, and a version. S3 takes a file, with versioning on the bucket.
- Output or effect: Prompt Management gives a version ARN the app can call. S3 gives an object version, and your code must load and fill the template.

Question to ask yourself: Should the managed service fill the variables, or will the team own that code?

Trap: File versioning keeps history but does not fill variables. The team then writes and tests the fill logic. S3 versioning details: Not in source.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/logging-using-cloudtrail.html and course slide extract page 10.

DRAFT versus numbered version

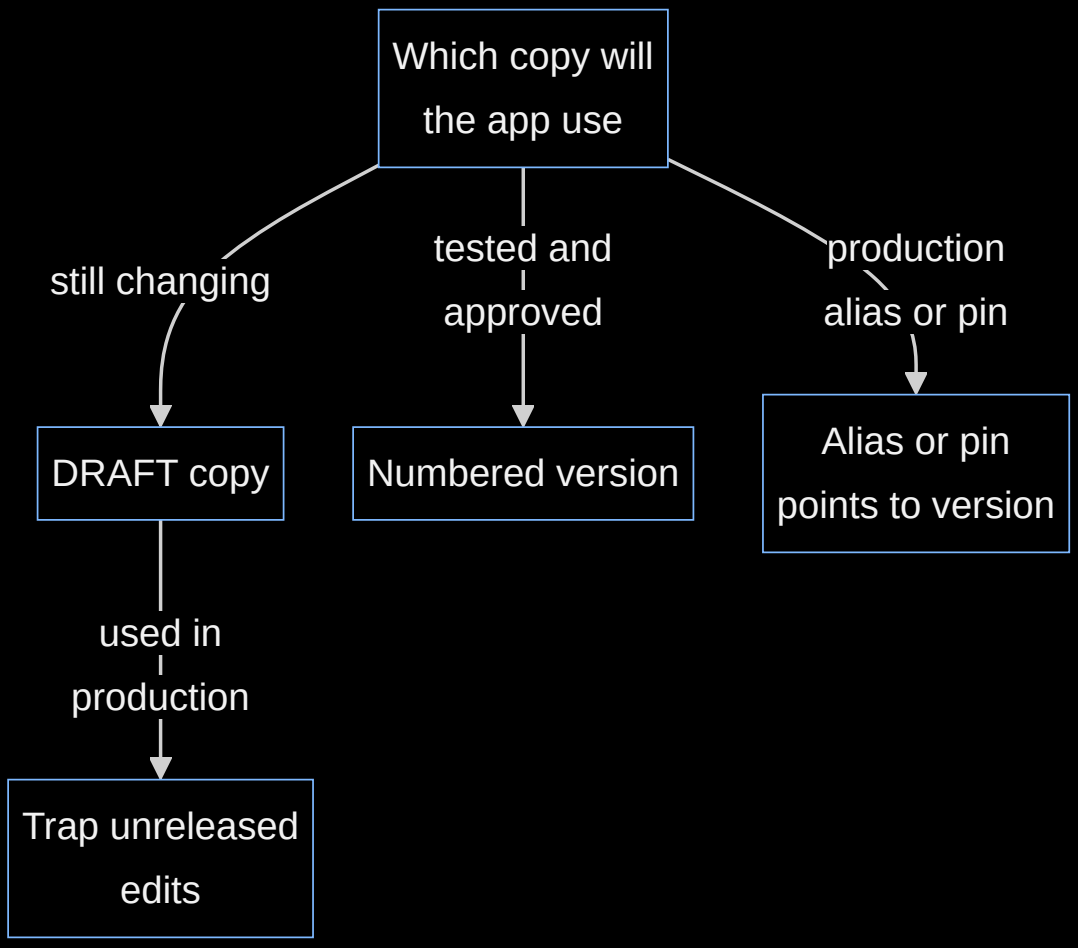


Figure: Only a numbered version or a version-pointed alias is a release.

- Purpose: DRAFT is the working copy. A numbered version is a fixed point that you can deploy or call.
- Input: DRAFT takes edits. A numbered version is created from a snapshot of the working copy.
- Output or effect: An agent alias pointed at a version keeps production steady while DRAFT changes. A guardrail pinned to a version does the same for safety rules.

Question to ask yourself: If someone edits this copy today, does production change?

Trap: Calling DRAFT in production sends unreleased edits to users. The docs say agent versions are immutable snapshots.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/deploy-agent.html and https://docs.aws.amazon.com/bedrock/latest/userguide/logging-using-cloudtrail.html. Guardrail immutability: Not in source.

Lambda alias weights versus CodeDeploy versus AppConfig experiment

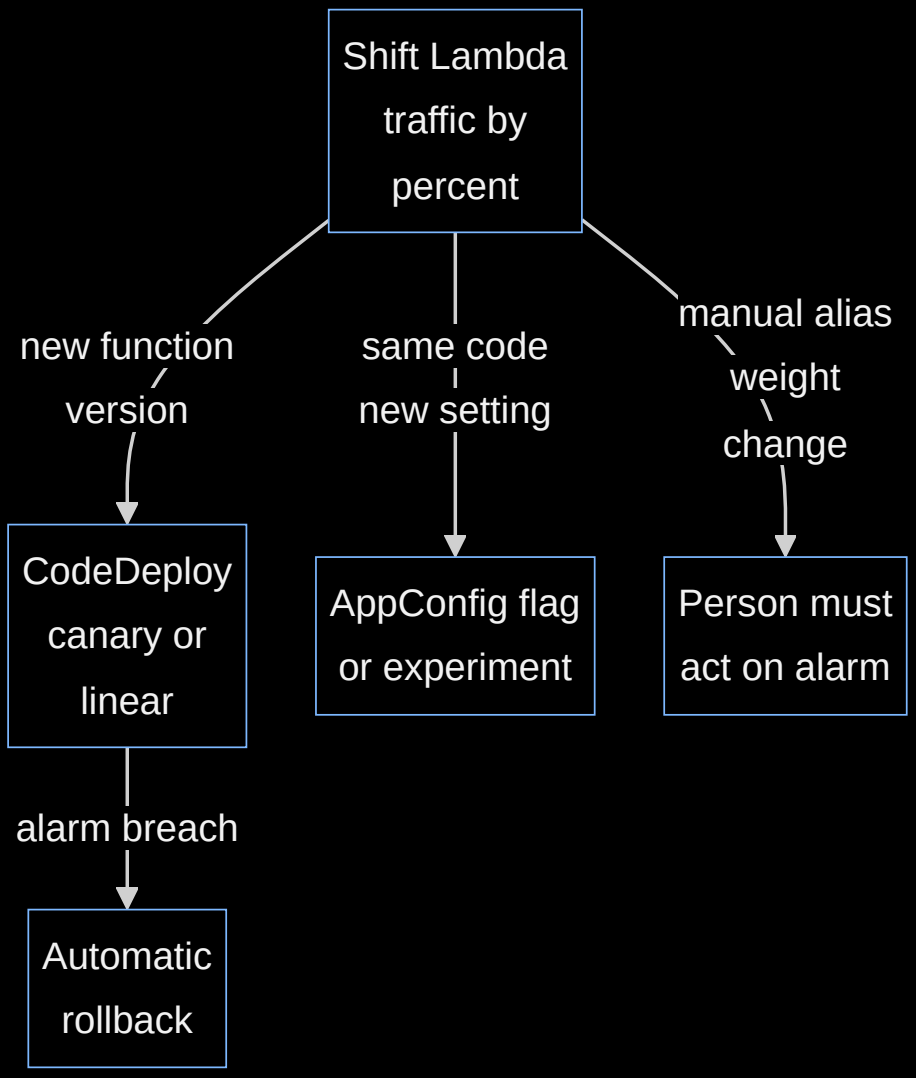


Figure: Code change, setting change, and manual shift need different tools.

- Purpose: Alias weights split traffic between two versions. CodeDeploy shifts alias traffic with rules and alarms. AppConfig splits traffic for flags and experiments with no code deployment.
- Input: Alias weights take a manual value. CodeDeploy takes a deployment configuration and alarms. AppConfig takes a flag, a target, and a percentage.
- Output or effect: A manual weight change waits for a person. CodeDeploy rolls back on alarm. AppConfig changes the setting and rolls back when configured.

Question to ask yourself: Is the change a new function version, or a new setting for the same code?

Trap: A manual alias weight change does not roll back by itself. The requirement for no manual step rules it out.

Source: course slide extract page 388 and https://docs.aws.amazon.com/appconfig/latest/userguide/what-is-appconfig.html

Single CloudFormation stack versus StackSet

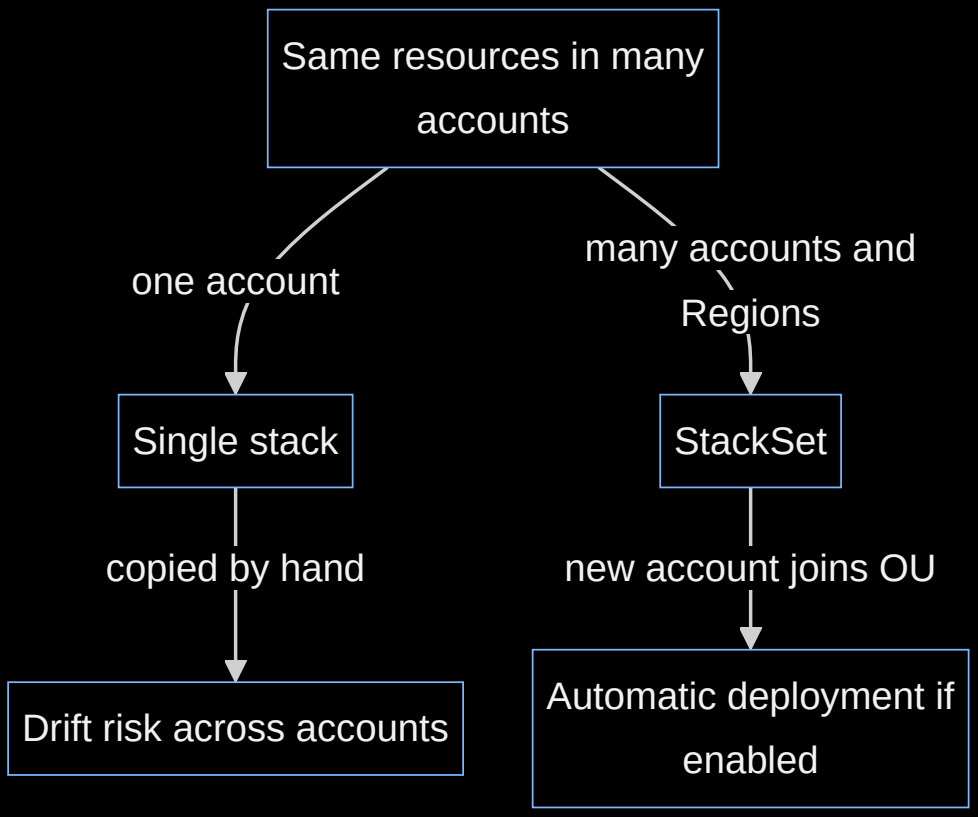


Figure: Repeat by hand is the wrong answer when no manual step is allowed.

- Purpose: A stack creates resources in one account and Region. A StackSet pushes one template out to many accounts and Regions.
- Input: A stack takes a template and parameters. A StackSet takes a template and a set of target accounts or an OU.
- Output or effect: A stack gives one set of resources. A StackSet gives one stack instance for each target.

Question to ask yourself: Must new accounts receive the same setup with no manual step?

Trap: Sharing a template for each team to run by hand does not meet a no-manual-step rule.

Source: course slide extract pages 577 and 585. StackSet details: Not in source.

AWS owned or managed key versus customer managed KMS key

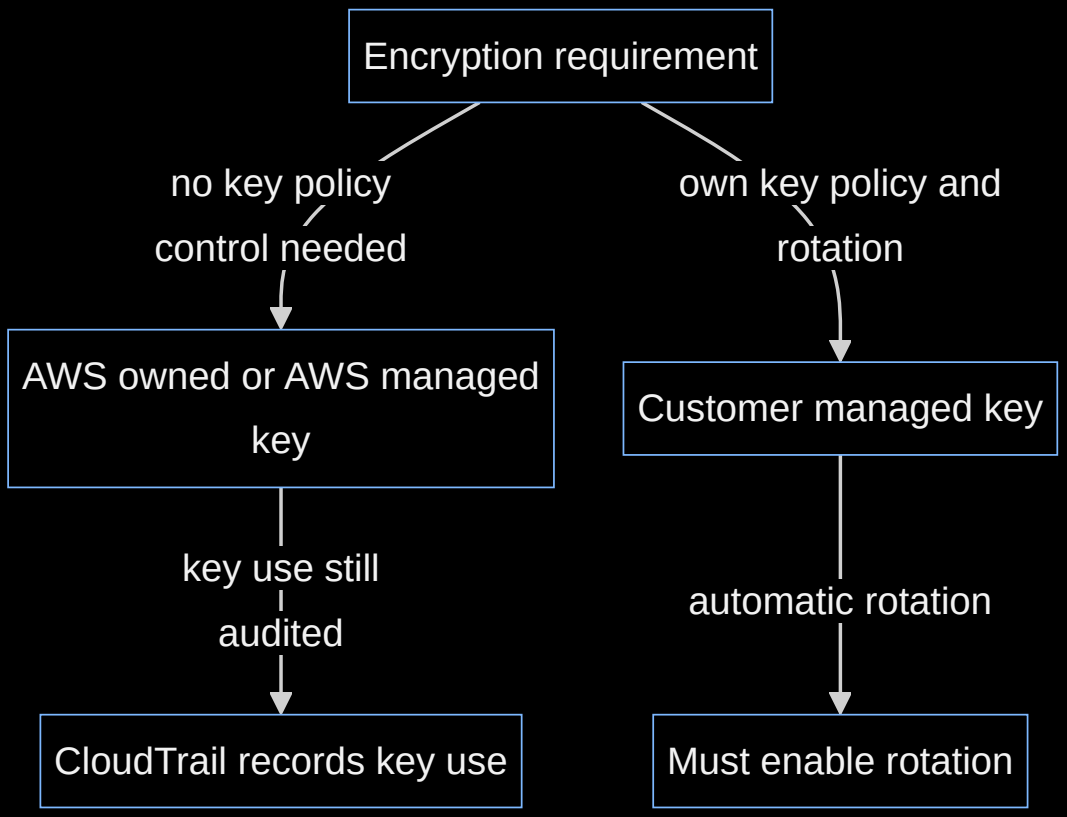


Figure: Control over the key policy decides the key type.

- Purpose: AWS owned and AWS managed keys are run by AWS. A customer managed key lets you set the key policy and rotation.
- Input: An AWS managed key takes the service default. A customer managed key takes a key you create, with a key policy.
- Output or effect: A default key encrypts with no key policy from you. A customer managed key gives control and carries a monthly fee on the slide.

Question to ask yourself: Must your team write the key policy or prove control of the key?

Trap: A default key does not satisfy a rule that needs your own key policy.

Source: course slide extract pages 495, 496, and 497.

Read a long question

Each worked scenario below uses three sentences. The constraint words are in bold. Each wrong option gets one reason. The answer comes last. Work through the scenario before you read the answer.

Worked scenario 1: exact text and a fast way back

A claims team changes the wording of a Bedrock assistant prompt. QA must test the exact text that production uses. Production must return to the last approved text in minutes, with no code build.

Exam tip: The bold words are **exact text**, **last approved**, **in minutes**, and **no code build**. Each one removes a type of option.

- A rebuild of the function from source is rejected. It needs a build, so it misses the no-build rule.
- Reading the latest object from S3 is rejected. Latest is not a fixed approved text, so QA and production can differ.
- Copying the QA draft into production by hand is rejected. It creates drift and leaves no release record.
- Storing an approved prompt version ARN in AppConfig, then moving the value back, is the answer. QA tests the same version ARN, and the value change is a rollback.

Rule: For exact and fast rollback, pin a version ARN and move a configuration pointer.

Source: course slide extract page 364 and https://docs.aws.amazon.com/appconfig/latest/userguide/what-is-appconfig.html

Worked scenario 2: who called, what was said, and where it is kept

A bank must show which caller invoked its model and when, for every request in a year. The compliance team also wants prompt and response text kept for only 30 days. All records must stay in the production account.

Exam tip: The bold words in this kind of question are the record type, the retention period, and the account. Match each one to a service before you read the options.

- Sending invocation logs to a central logging account is rejected. Invocation log destinations sit only in the configuring account and Region.
- CloudTrail alone is rejected. It shows the caller and time, but not the text.
- X-Ray traces are rejected. They show timing and the call path, not the caller record or the text.
- CloudTrail with a trail to an S3 bucket in the production account, plus invocation logs with 30-day retention on the destination, is the answer.

Rule: Keep caller records in CloudTrail and content records in invocation logs. Set retention on each destination.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/logging-using-cloudtrail.html and https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html

Worked scenario 3: private subnets, agent calls, and one role

A healthcare app runs in private subnets with no internet route. It calls a model directly and also calls an agent. Security wants only the app role to use the private path, and the hostnames must stay the standard Regional names.

Exam tip: The bold words here are no internet route, the agent call, standard hostnames, and only the app role. The agent call is the detail that removes an answer that covers only one endpoint.

- A NAT gateway is rejected. It gives the private subnets internet access.
- A gateway endpoint for Bedrock is rejected. The Bedrock endpoint page lists only interface endpoints.
- One interface endpoint for bedrock-runtime is rejected. It misses the agent runtime calls.
- Interface endpoints for bedrock-runtime and bedrock-agent-runtime, with private DNS and an endpoint policy for the app role, is the answer.

Rule: One Bedrock API surface needs its own interface endpoint. Private DNS keeps the hostname. The endpoint policy limits the role.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/vpc-interface-endpoints.html and course slide extract pages 518 and 522.

Distractor patterns seen in the tests

Wrong options follow a small number of patterns. Learn the patterns first. Then each new question becomes a check against a known pattern. Each pattern below names the pattern and says why it is wrong.

- **Right service, wrong scope.** The option names a real feature but applies it to one account when the need is one environment, or to one environment when the need is one account. Why wrong: the scope does not match the requirement.
- **Latest instead of pinned.** The option reads the newest prompt, guardrail, or object. Why wrong: production then changes without a release decision.

- **Metric used as evidence.** The option proposes a dashboard as the audit record. Why wrong: metrics count events and do not hold the content or the caller record.
- **Logs used as a control.** The option proposes logs to stop a bad output. Why wrong: logs record a call. Guardrails or IAM stop it.
- **Custom code for a managed feature.** The option builds a router, a registry, or a fill engine. Why wrong: when the question asks for least operational overhead, managed features win.
- **Internet detour labeled private.** The option uses a NAT gateway or a public route. Why wrong: any path to the public internet fails a no-internet rule.
- **Wrong endpoint type.** The option uses a control plane endpoint for runtime calls, or a gateway endpoint for Bedrock. Why wrong: each Bedrock API surface needs its own interface endpoint.
- **Manual step for an automatic rule.** The option waits for a reviewer or a person to press a button. Why wrong: the question asks for no manual step.
- **Counting the wrong event.** The option counts stream completion events as calls. Why wrong: those events are not API calls, and they double the count.
- **Shared secret or data source.** The option gives dev and production one secret or one data source. Why wrong: a test can then reach live data or live credentials.

Trap: A correct-sounding option that uses a real service can still fail on scope. Check the scope words before you check the service name.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/logging-using-cloudtrail.html, https://docs.aws.amazon.com/bedrock/latest/userguide/vpc-interface-endpoints.html, and course slide extract page 520.

Drill

Read the question line. Cover the answer line. Say the answer aloud, then check it.

1. QA must test the exact prompt text that production will use, with fixed variables.

Rule: Test a Prompt Management version ARN, not the working copy.

2. Production must not start a stage until an audit gate has passed.

Rule: Use an Entry condition with the Fail result.

3. A failed production deployment must restore the last good revision with no person acting.

Rule: Use an On Failure condition with the Rollback result.

4. A team wants a failed audit gate to look like a normal skip.

Trap: Skip ends the stage quietly, so it hides the failed gate.

5. Turn a feature on for five percent of users, with validation first and no deployment.

Rule: Use AppConfig with a validator and a deployment.

6. A database password must rotate on a schedule without a code change.

Rule: Use Secrets Manager with a rotation function.

7. Each environment needs a model ID that the app reads at startup, with no rotation.

Rule: Store the value in Parameter Store per environment.

8. An auditor must see who called Bedrock and when, across the year.

Rule: Use CloudTrail with a trail that delivers to S3.

9. The compliance team must keep prompt and response text for investigations, in the same account and Region.

Rule: Turn on invocation logs that land in CloudWatch Logs or S3 for that account and Region.

10. A central logging account must receive invocation logs from several workload accounts.

Trap: Invocation log destinations sit only in the configuring account and Region.

11. A private subnet app must call the agent runtime, with no internet route.

Rule: Use an interface endpoint for bedrock-agent-runtime with private DNS.

12. Stop every member account from using a service that policy forbids.

Rule: Use a service control policy on the OU that holds the accounts.

Sources

AWS documentation pages used in this module. Each one was fetched with WebFetch.

- https://docs.aws.amazon.com/bedrock/latest/userguide/deploy-agent.html

- https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring-runtime-metrics.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring-runtime-otps.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/vpc-interface-endpoints.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/logging-using-cloudtrail.html
- https://docs.aws.amazon.com/appconfig/latest/userguide/what-is-appconfig.html
- https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.h
- https://docs.aws.amazon.com/codepipeline/latest/userguide/stage-conditions.html

Source: course slide extract for this module, pages 10, 109, 210, 211, 281, 314, 364, 374, 376, 377, 383, 388, 389, 448 to 451, 454, 459, 461, 477, 484, 489, 495 to 497, 500, 501, 518, 520, 522, 577, 578, 585, and 619.